A6: Satellite Buddy

Anna Nguyen, Braden Lee, and Stephen Chien

Department of Electrical and Computer Engineering, Carnegie Mellon University

***Abstract*— Satellite Buddy is a portable, Internet-independent weather station that intercepts and displays real-time weather information from Geostationary Operational Environmental Satellites (GOES). Existing solutions either require broadband connection, rely on host PCs for software-based decoding, or traditional stations cost tens of thousands of dollars. Our self-contained embedded system does not require Internet and offloads BPSK demodulation and HRIT decoding to two dedicated FPGAs, which eliminates the need for an external PC, reducing end-to-end processing latency to under 30 ms. Furthermore, the system operates on a standard 5V battery and weighs under 5 lbs, improving accessibility and portability.**

***Index Terms* – Radio Signals, Signal Processing, SDR, FGPA, Hardware Acceleration**

I. Introduction

Whether to track the trajectory of a nearby storm system or simply ascertain whether to pack an umbrella, weather forecasts indubitably affect the ways we live. Nowadays, weather predictions are broadly disseminated to the general public via the Internet, which allows users to receive it on their phones, laptops, web browsers—anything with an online connection. However, this coverage excludes offline communities, like those living in remote residences or digital deserts. Although emergency weather warnings are broadcast over radio, access to a comprehensive daily forecast provides incredible utility beyond awareness of solely extreme circumstances. For example, knowing to expect precipitation allows people to plan against inclement conditions.

The Geostationary Operational Environmental Satellite (GOES) system is a series of NOAA operated satellites equipped with meteorological instruments for weather forecasting. In particular, these satellites coordinate with continental-based stations to generate timely weather forecasts distributed as images. Notably, anyone can receive the broadcast without charge and without any Internet. Our project aims to create a portable weather station that receives signals from these satellites to provide real-time weather information to users. Our system operates without any Internet connection, making it much more reliable and accessible during disasters or in remote areas. Moreover, our system can be built entirely using commodity hardware, which is much cheaper compared to traditional weather stations which cost tens of thousands of dollars to build, setup, and operate.

II. Use-Case Requirements

Our system, first and foremost, must be able to receive high quality signals from the GOES satellite. Specifically, we are aiming for a signal-to-noise ratio (SNR) of ≥ 25 dB, achieved via high-gain directional antenna of ≈ 21 dBi and a low-noise filtered L-band amplifier designed for GOES reception, which will provide ≥ 20 dB active gain within the GOES L-band (~1688 MHz). By ensuring clear reception and amplification of the GOES signal, we minimize corruption in the bitstream we receive and reduce packet losses, which both lead to higher quality and timely weather data for users.

Secondly, our system must provide ease-of-use for the end-users. Specifically, our system will perform all processing of data locally without cloud services, i.e. RF front end connects directly to the FPGA and the Raspberry Pi. This increases the reliability and accessibility of our system, enabling users and communities without Internet or broadband access to receive high-quality, real-time weather reports and alerts.

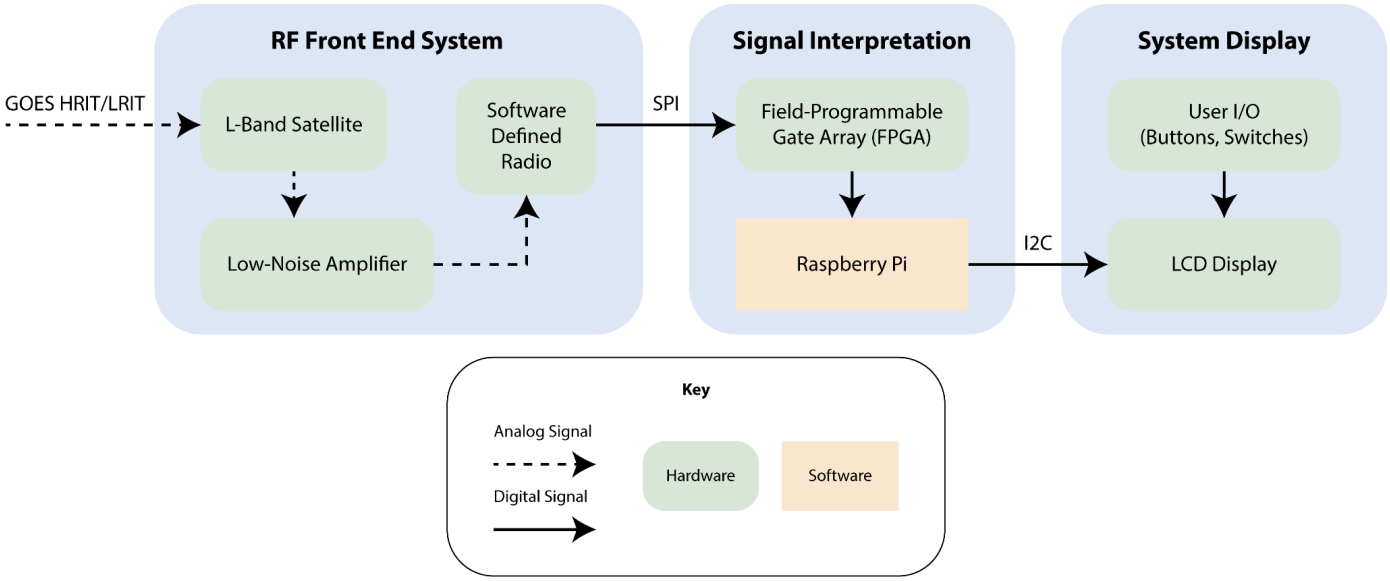
Moreover, we aim for a < 30ms end-to-end latency between the RF front- end receiving the GOES signal to displaying the information on the user-facing screen, achieved by pipelining and parallel processing of signals in our FPGA. We target < 200ms latency from the user touching the screen to performing the desired action, achieved by using a lightweight GUI, OS, and device drivers optimized for performance.

Finally, we target a total current draw of ~500 mA, which allows our system to be powered via standard 5V 5000mAh li-ion batteries, improving the portability of the system. While our system cannot be carried in a backpack due to the size of the antenna, the total weight of all components is kept under 5 lbs, making it easy to be carried around by one single person.

III. Architecture and/or Principle of Operation

To receive data from GOES-19, a parabolic dish antenna is necessary to capture RF signals; specifically, the LRIT/HRIT protocol is broadcast in the L-band at 1694.1 MHz. The acquired raw signal is fed into a low-noise amplifier (LNA). The LNA improves the receiver's signal-to-noise ratio by boosting low-power signals without further distorting the signal. This enables the system to process satellite communications even when the signal is weak. The amplified signal is passed to a software defined radio (SDR) for processing; here, the signal is filtered and demodulated to translate the analog signal into digital data. Specifically, the SDR denotes a sampled point on the signal in I/Q form, representing it as a complex vector.

After the data is digitally represented as I/Q data, it is transferred to the FPGA for protocol decoding. The GOES signal targeted by this system is modulated with BPSK and adheres to the LRIT/HRIT protocol. Complying with the specifications for these protocols, the FPGA incrementally decodes the digital signal into an weather forecast image. As each image frame is generated, the decoded packets are transferred to the Raspberry Pi. Upon receiving all packets composing an image frame, the Raspberry Pi assembles the image and transmits it to an external display via I2C.



The LCD display communicates with the Raspberry Pi to update itself when new weather forecasts are available. It also interfaces with user I/O components, i.e. buttons, switches, for users to toggle between historical forecasts and instrument filters.

Fig. 1. The same full block diagram as in Fig. 2 is placed spanning both columns.

IV. Design Requirements

The foremost necessity for this device to operate as specified is the ability to operate without an Internet connection. Hence, all processing must occur locally without reliance on cloud services. Although the Raspberry Pi includes Wi-Fi compatibility, our device should function independent of it, entailing, in part, that any requisite libraries and dependencies should be downloaded onto the device.

The GOES system was chosen for their geostationary status, meaning weather data can be continuously streamed throughout the day without having to adjust the antenna bearing. Furthermore, the GOES array provides signal coverage throughout North America. Subsequently, users should receive real-time weather updates. In the context of our system, we define real-time updates as a maximum of 30 ms end-to-end latency. To interface with the weather satellites, the design must include capabilities to receive, demodulate, and interpret signals broadcast at 1694.1 MHz, modulated with binary phase shift keying, and encoded as LRIT/HRIT.

The system should also – when environmentally possible, i.e. antenna unimpeded and accurately oriented – ensure the receiver reliably captures signals to minimize packet corruption; hence, the RF front-end, in combination with a high-gain directional antenna and low-noise amplifier, should ensure a minimum signal-to-noise ratio of 25 dB. Furthermore, as it will be communicating with satellites 22,000 miles above the equation, the design should be prepared to handle weak signals. The system should provide a minimum of 20 dB active gain to low-power signals without distorting them beyond recognition.

The design also includes I/O elements for user interaction. Any intended interaction with the system should produce a noticeable effect within a recognizable timeframe; specifically, within 200 ms from interaction to display. Comprehensively, the design should be prepared to operate on a lightweight, rechargeable battery. Users should expect a current draw around 500 mA, such that it could be powered via a standard 5V 5000mAh li-ion.

V. Design Trade Studies

This section derives the design equations that govern the modulation scheme, FPGA choice, and antenna size. Each trade study links a design specification to one or more use case requirements.

A. Modulation Scheme: BPSK vs. QPSK

One design decision is the choice of modulation used to demodulate the GOES HRIT downlink. The GOES HRIT signal uses BPSK, which has several advantages for the design over QPSK.

The bit error rate for BPSK is:

$$\text{BPSK } P_b = Q\left(\sqrt{\frac{2E_b}{N_0}}\right) \quad (1)$$

For QPSK the bit error rate is:

$$\text{QPSK } P_b = Q\left(\sqrt{\frac{2E_b}{N_0}}\right) \quad (2)$$

Equations (1) and (2) show that, in theory, BPSK and QPSK achieve identical BER. However, practical implementation loss for QPSK is greater because the carrier phase recovery must resolve a 4-symbol constellation which requires roughly 9–12 dB, whereas the BPSK 2-symbol constellation requires

roughly 6–8 dB.

Furthermore, since BPSK only uses two phases rather than four phases (QPSK), it requires less logic and hardware on the FPGA. This reduction in logic is appealing for this project as the goal is to demodulate purely through hardware on the integrated FPGA on the selected SDR.

B. FPGA Board Selection: DE10-Standard

The use case requirement is to receive and reconstruct image files transmitted over the GOES HRIT downlink in real time. This drives the design specification that the FPGA must implement the complete HRIT protocol processing pipeline. This pipeline primarily consists of five stages (sync marker detection, CVCDU derandomization, forward error correction, VCDU virtual channel filtering, M_PDU reassembly). Combining all five stages, the total DSP block requirement is dominated by the FEC and derandomization modules as they are the most math intensive stages. Here is a table comparing the FPGAs that were considered:

Table I: FPGA

Criteria	**DE10 Stanadard**	**Ultra 96-V2**	**DE0 Nano**
DSP Blocks	112	360	16
On-Board SDRAM	64 MB	2 GB	32 MB
Satellite/SDR Use	Yes	No	Yes
Quartus Support	Yes	No	Yes

The DE10 Standard was ultimately selected because it provided enough DSP block and on-board SDRAM needed for the project. It has also proven community use in open-source satellite projects, and is fully supported by the Quartus Prime toolchain which is a tool the team is comfortable and used to using.

C. Antenna Sizing (21 dBi)

In order to receive the GOES HRIT downlink, an antenna that achieved the required gain and was small enough to move to a roof top or open field. To select an antenna, the following criteria were evaluated:

Table II: Antenna

Criteria	**Noolec**	**Bolton**
dBi	21	28
Price	99.95	199.99
Weight	3.4 lbs	5.3 lbs

While the Bolton antenna had a higher dBi, the Noolec antenna was ultimately chosen as it was cheaper and had a high enough gain for GOES reception. Furthermore, the Noolec antenna was lighter than the Bolton which aligned with the use case requirement of portability.

VI. System Implementation

We now describe the system implementation details. As seen from the architecture diagram above, our system can be split into 3 parts: RF frontend, signal processing in FPGA, and data display in software. We will describe each of these in detail.

A. RF Frontend

Our RF frontend consists of a parabolic dish antenna, which receives GOES signals. As GOES is geostationary, we are able to receive signals from it all 24-hours. However, depending on the time of the day, we do need to adjust the azimuth and tilt of the antenna to ensure we have a clear line-of-sight with the GOES satellite. Our setup is as follows:



Fig.2. The antenna in operation at The Mall

As seen in the image above, our setup consists of the dish antenna, a metal piece connected to the center of the antenna to reflect signals back to the dish, along with a tripod to keep the antenna fixed at the correct azimuth and tilt, and to prevent external factors (for example, wind) from displacing/ toppling the antenna.

Once GOES satellite signal is received by the parabolic dish antenna, we connect to an LNA that provides the required gain as outlined in the user and design requirements. We then feed the amplified signal from the LNA to SDR (LimeSDR) which converts the analog signal into I/Q samples (which represents the quadrants of the digital signal). The LimeSDR comes with an embedded small FPGA which we synthesize a demodulator onto, converting the I/Q samples into a binary stream. We use a USB-to-JTAG converter to program this embedded FPGA from our laptop.

B. Signal Processing

This module takes the raw bitstream from the FPGA on the SDR and converts it into full files containing image data that are transmitted by the GOES satellites. The main components are all implemented using the DE-10 Standard FPGA board to take full advantage of parallelism and low-latency of hardware (instead of using the slower alternative: software processing). The final component of assembling file segments into full files is done in software, as it has much more memory to work with and speed is not crucial once we have done most of the signal processing.

A functional diagram of the hardware components is shown below:

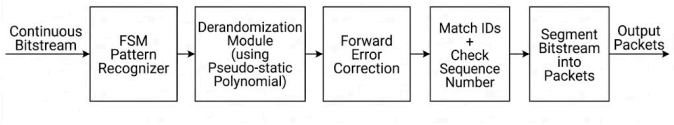


Fig.3. Functional diagram of hardware modules.

As seen in the figure from left to right, the raw bitstream is fed into an FSM that synchronizes against a fixed bit pattern of 32 bits, then listens in on the next 8192 bits. Next, we have a module that derandomizes the 8192 bits by multiplying each individual bit with an output bit generated by a pseudorandom static polynomial (implemented using shift registers). We then perform forward error correction (FEC) using Reed-Solomon encoding.

We then check the ID in each header and discard frames that don't match the ID of the GOES satellite. We also check the sequence number of each frame to ensure that we receive frames in-order. Each frame contains multiple packets, which we extract with the last FSM, and send each individual packet to our software running on the Raspberry Pi. These modules are all implemented in SystemVerilog, using Synopsys VCS for simulations and Quartus to perform synthesis.

C. System Display

The software running in the Raspberry Pi will perform three main functions. Firstly, it will reconstruct files (following the standard outlined by LRIT/HRIT specifications, see [2]) from packets received from the FPGA board. Next, it will generate a time- stamp (using the system clock) and archive these image files in system storage. Finally, it will respond to device interrupts from the touch screen and change the type/date of images that are displayed on the screen. This is represented by the functional diagram below:

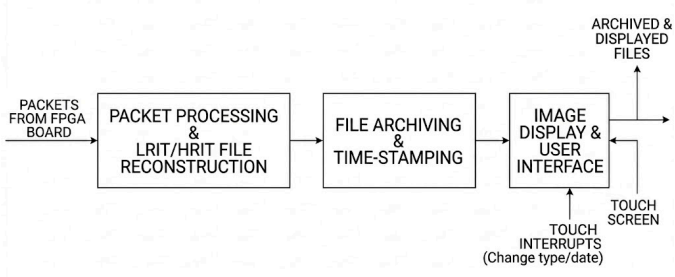


Fig.4. Functional diagram of software modules.

VII. Test, Verification and Validation

This section covers the tests that the team will run to ensure that the Satellite Buddy will meet the specified use-case requirements.

A. *End-to-End Latency*

We will measure the latency of the end-to-end system and ensure we are able to display up-to-date weather information within 30 seconds of powering up the device and receiving data from the satellite using a stopwatch. This test will be repeated 10 times and the maximum latency observed must remain at or below 30 seconds. This confirms that the user will see the data in a timely manner.

B. *Display Responsiveness ($< 200ms$)*

We will measure the responsiveness of the device by ensuring that users are able to switch display information (for instance, from visual to infrared images) within 200ms. To measure this, we can use an oscilloscope to measure when the screen is triggered and measure the time between the trigger and the updated display. The reasoning for choosing $< 200ms$ response time is because this is the threshold below which humans can perceive a response as instantaneous [2].

C. *Internet Independent Operation*

To validate that the system relies solely on direct satellite reception, the device will be operated in an environment with all Wi-Fi, cellular, and Ethernet connections physically disabled or blocked. Correct reception and display of live weather imagery in this condition confirms that no internet or cellular infrastructure is required, validating the use-case requirement of internet independent operation.

D. *RF Front-End SNR ($\geq$ 25 dB)*

The SNR of the RF front-end will be measured using a spectrum analyzer connected directly to the front-end output. With the antenna pointed at the GOES satellite, we will record the signal power and compare it to the noise floor. A result of 25 dB or above confirms that the front-end provides sufficient signal quality for reliable frame synchronization and demodulation.

E. *LNA Gain ($\geq$ 20 dB) & FPGA Input Voltage ($\geq$ 1.2 V)*

The gain of the LNA will be verified by measuring input and output signal power with a spectrum analyzer. In addition, a multimeter will confirm that the voltage level at the FPGA data input meets the 1.2 V minimum logic-high threshold. These two tests together confirm that the signal chain amplifies the weak satellite downlink sufficiently before digital sampling, preventing bit errors caused by insufficient drive level.

F. *Battery Life ($\geq$ 2 Hours on 5 V / 5000 mAh Li-ion)*

The fully assembled device's power consumption will be measured using a multimeter to record the current draw while the device operates under normal conditions. Total power will be calculated using the known supply voltage of 5 V. This value will be compared against the battery specifications to verify the design operates within acceptable limits and to project expected runtime on a given battery capacity.

VIII. Project Management

This section specifies the project schedule, division of labor between the team members, bill of materials, and risk mitigation plans.

A. Schedule

The primary milestones for the Satellite Buddy are the RF front-end, data processing back-end, and the user interface and display system.

The RF front-end receives the GOES-19 signal and passes the data to the data processing back-end. The tasks that need to be accomplished in this section are understanding the I/Q format, capturing clear satellite data, and demodulating the I/Q data with the FPGA on the SDR.

The data processing back-end takes the satellite data and decodes the protocol. The primary task that needs to be accomplished in this section is designing a RTL to determine and manage the file sections that are to be sent to the Raspberry Pi.

Lastly, the user interface and display system takes the file sections from the FPGA and reconstructs the file to display the weather forecast on a touchscreen display for the user to interact with.

B. Team Member Responsibilities

Anna is primarily working on the data processing back-end. She is responsible for designing the RTL for decoding the protocol. Her focuses are the sync marker, VDCU handler, and M_PDU handler modules. This includes the verification and testing of the RTL pipeline as a whole. Her secondary task is the RF front-end, where she will work with Braden and Stephen to get sample data to feed into the FPGA backend.

Braden is primarily responsible for the RF front-end. He will be leading the data collection from the GOES, with the use of GNU Radio, as well as the demodulation of the I/Q data received, which is done with the FPGA on the SDR. His secondary task will be the FEC module in the protocol decoder.

Stephen is primarily responsible for the user interface and display system. This includes reconstructing the file sections from the protocol decoder and sending/receiving data to/from the touchscreen display with the Raspberry Pi. His secondary task is the derandomization module and collaborating with Anna on the M_PDU handler.

All team members participate in weekly meetings and system testing. There are many overlapping sections as the RF front-end and data processing back-end are the most work intensive.

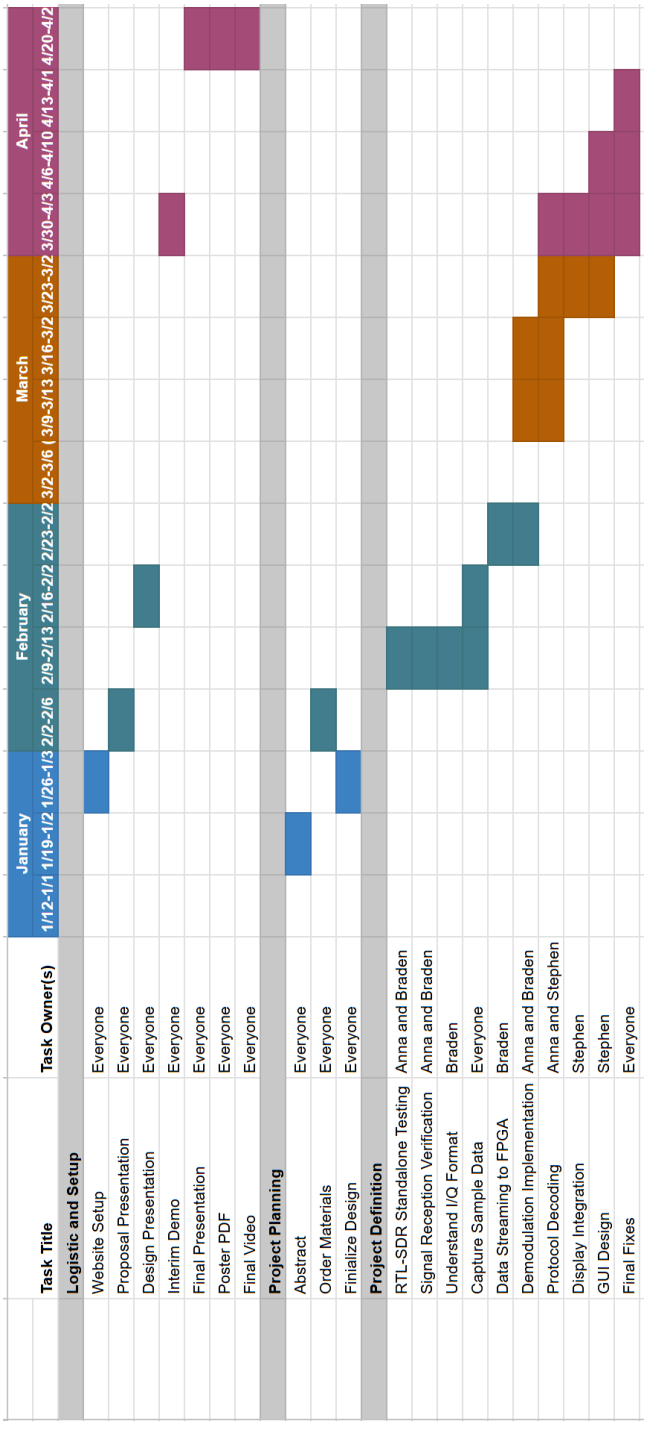


Fig. 2. Schedule with milestones and team responsibilities

C. Bill of Materials and Budget

See Appendix at the end of the document.

D. TechSpark Use Plan

We do not plan on using TechSpark for our project.

E. Risk Mitigation Plans

The main risks of this project are signal reception and FPGA resource constraints, both of which could cause the system to fail to meet its primary use case requirement of reliable, real-time image reception and display.

Firstly, no team member has had previous experience with deploying an antenna for signal reception. In practice, obstruction or non-ideal LNA noise may prevent the antenna from receiving a strong enough signal, causing the FPGA to fail in properly demodulating/decoding the protocol. To account for this the team has identified the following mitigations: (1) relocate to a more open space to remove any obstructions to the antenna (2) replace the RF front-end LNA with a one with lower noise figure (3) upgrade to a higher gain antenna (> 21 dBi).

Secondly, with the large amount of data being sent to the system, the chosen FPGA might not be able to handle decoding the protocol fast enough, causing us to drop packets, which will corrupt the image data. To account for this, the file reassembly has already been moved to the RPi instead of handled in the FPGA. Further mitigations could include moving more complex or stateful decoding logic to software, while keeping the parallelizable operations on the FPGA.

IX. Related Work

There are several existing projects that address overlapping aspects of the Satellite Buddy design. For instance, SatDump is a general-purpose, multi-platform satellite data processing application supporting live decoding of GOES HRIT over 90 additional satellite links, compatible with RTL-SDR, Airspy, HackRF, LimeSDR, and PlutoSDR devices across Windows, macOS, Linux, and Android. SatDump executes the full pipeline in software on a host CPU and provides a graphical imagery display. Unlike the Satellite Buddy, it requires an attached PC, making it a software-only solution rather than an integrated embedded system. Many other projects follow a similar design where they rely entirely on a CPU-based software decoding without a dedicated hardware decode path.

Satellite Buddy distinguishes itself by integrating the full pipeline from RF reception through FPGA-based HRIT decoding to image display in a single self-contained portable platform, allowing everything to be self-contained and independent from external servers.

X. Summary

The Satellite Buddy is a portable, independent GOES HRIT ground station that receives live weather satellite imagery without a host PC. It combines a 21 dBi antenna, LNA, and SDR front-end, a DE10-Standard FPGA that handles the full HRIT decode pipeline in hardware, and a software display front-end. The key design insight is that moving BPSK demodulation and protocol decoding into FPGA frees the microprocessor for the stateful reassembly work, making the whole system run self-contained.

For stakeholders who care about the use case, hobbyist weather enthusiasts, and emergency managers in areas with unreliable internet, the impact is access to real-time GOES imagery without depending on NOAA's web servers or a desktop workstation. The system is designed to be set up on a rooftop or tripod, pointed towards the satellite, and able to run.

The two biggest implementation challenges are the FPGA to RPi communication interface and RF signal integrity at the front-end.

On the hardware communication side, the FPGA needs to hand off file sections to the RPi to reassemble and display. The challenge is that this interface has to be fast and reliable enough that the Pi does not fall behind. If the receive buffer on either side fills up, frames are silently dropped and the file segments break, corrupting whatever image file was being reconstructed. Getting the protocol right for this handoff and making sure the RPi's software can drain it fast enough under load are challenges we need to face.

On the RF front-end side, the challenge is that there is no good way to know before deployment whether the received signal will be clean enough for the demodulator to properly work. Below a certain SNR threshold the BPSK demodulator simply fails to acquire frame lock and the system produces nothing. This makes the RF front-end the most fragile part of the system to validate, since diagnosing it requires actually pointing the antenna at the satellite and measuring the received signal quality.

Glossary of Acronyms

BPSK – Binary Phase Shift Keying
CVCDU – Coded Virtual Channel Data Unit
DSP – Digital Signal Processor
FPGA – Field Programmable Gate Array
GOES – Geostationary Operational Environmental Satellites
HRIT – High Rate Information Transmission
LNA – Low Noise Amplifier
M_PDU – Multiplexing Service Protocol Data Unit
NOAA – National Oceanic and Atmospheric Administration
QPSK – Quadrature Phase Shift Keying
RF – Radio Frequency
RPi – Raspberry Pi
RTL – Register Transfer Level
SDR – Software Defined Radio
SDRAM – Synchronous Dynamic Random-Access Memory
SNR – Signal to Noise Ratio
VCDU – Virtual Channel Data Unit

References

[1] Coordination Group for Meteorological Satellites LRIT/HRIT Global Specification, www.cgms-info.org/wp-content/uploads/2021/10/pdf_cgms_03.pdf. Accessed 28 Feb. 2026.

[2] Nielsen, Jakob. “Response Time Limits: Article by Jakob Nielsen.” Nielsen Norman Group, 1 Jan. 1993, www.nngroup.com/articles/response-times-3-important-limits/.

[3] Technologies, Terasic. “De Boards - Cyclone - DE10-Standard Development and Education Kit.” Terasic, www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=165&No=1081. Accessed 27 Feb. 2026.

Appendix (Price List):

Name	Link	Price	Ordered?
NooElec Sawbird LNA	https://www.ama	$40	Yes
1.7GHz GOES Antenna	https://www.ama	$100	Yes
5" Touchscreen	https://www.adaf	$75	Yes
Antenna Tripod	https://www.ama	$60	Yes
JTAG USB Programme	https://www.ama	$28	Yes
USB-A to C Adapter	https://www.ama	$10	Yes
Magnetic Compass	https://www.ama	$5.50	Yes
LimeSDR Mini V2.2	Inventory	$0.00	Yes
DE10 Standard	Inventory	$0.00	Yes
RPi 5 8GB	Inventory	$0.00	Yes
	Total:	$319	